

Complaint Intelligence Engine

Unsupervised NLP-Driven Complaint Discovery for Indian E-Commerce

M.Sc. Data Science — Unsupervised Machine Learning Course Project

NMIMS NSoMASA, Mumbai

Data Sources:

Live Google Play Store reviews (Myntra, Meesho, Nykaa, Flipkart, Amazon India) + Reddit PRAW (r/IndianShopping)

1. Project Overview

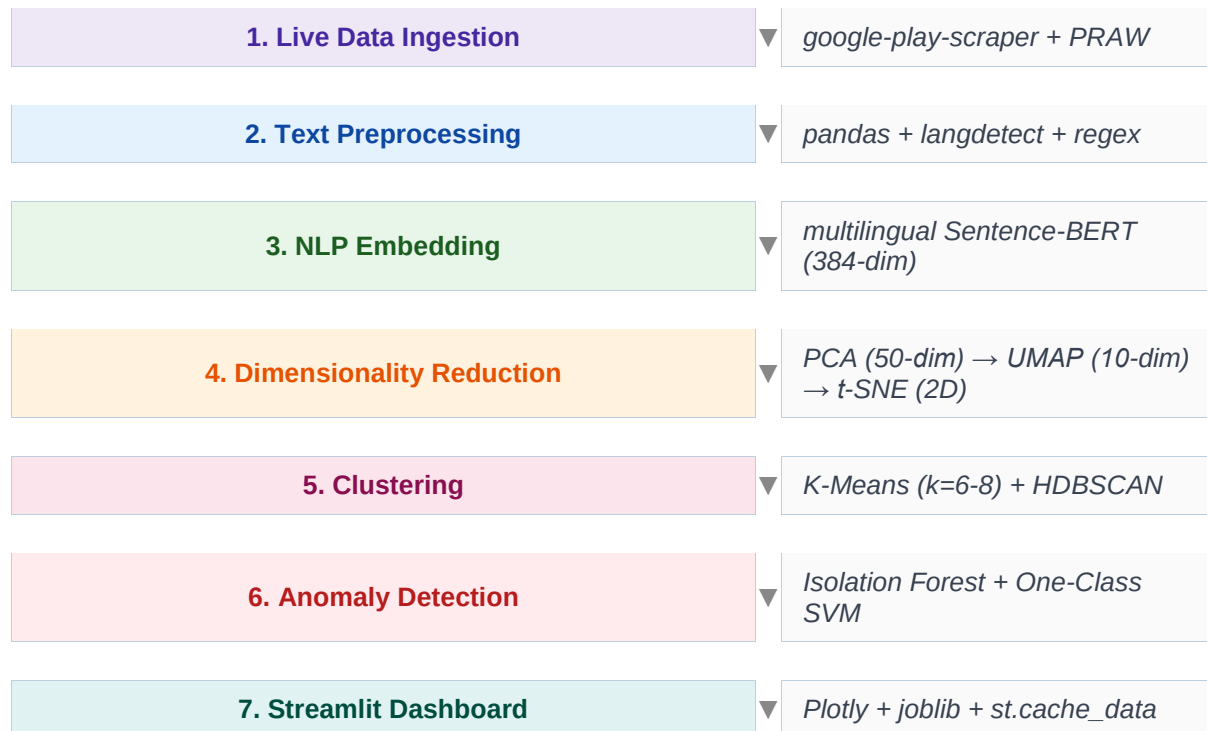
Indian e-commerce is experiencing a trust and quality crisis. Platforms like Myntra, Meesho, Nykaa, Flipkart, and Amazon India receive millions of user reviews documenting complaints about fake products, delivery failures, payment fraud, app crashes, and return policy violations. Yet no automated system exists to continuously monitor, categorise, and surface critical complaint patterns in real time — without any labelled data.

This project builds a Complaint Intelligence Engine: a fully unsupervised NLP pipeline that ingests live reviews from five major Indian e-commerce apps, encodes them using multilingual Sentence-BERT embeddings, discovers complaint archetypes through clustering, visualises the complaint landscape through dimensionality reduction, and surfaces anomalous critical complaints through Isolation Forest and One-Class SVM — all deployed as a live-refreshing Streamlit dashboard.

Key Question: Which Indian e-commerce platform is failing its customers right now, in which specific way, and how severe is it — with zero labelled data?

2. Project Architecture & Workflow

The pipeline consists of seven sequential stages. Each stage feeds directly into the next, with intermediate artifacts saved to disk so no stage needs to be recomputed from scratch.



The two data streams (Google Play and Reddit) are merged at Stage 1 into a single unified corpus. From Stage 3 onwards, all processing operates on the Sentence-BERT embedding space rather than raw text, making the pipeline language-agnostic and Hinglish-aware.

3. Step-by-Step Methodology

Step 1: Live Data Ingestion

The data pipeline collects live, real-time review data from two complementary sources: the Google Play Store (structured, star-rated, timestamped) and Reddit (unstructured, discussion-heavy, intent-rich). Both are free, require no commercial API keys, and refresh every time the pipeline runs.

1a. Google Play Reviews

- Use the google-play-scraper Python library (MIT licensed, no authentication required) to pull the 500 most recent reviews per app in both English and Hindi
- Target apps: Myntra (com.myntra.android), Meesho (com.meesho.supply), Nykaa (com.fsn.nykaa), Flipkart (com.flipkart.android), Amazon India (in.amazon.mShop.android.shopping)
- Each review record contains: reviewId, userName, content, score (1-5 stars), thumbsUpCount, timestamp (at), appVersion, and platform label
- Pull separately with lang='hi' to capture Hinglish reviews written in romanised Hindi — these are common among tier-2 city users of Meesho and Flipkart

1b. Reddit Supplementary Data

- Use PRAW (Python Reddit API Wrapper) to pull the top 200 posts from r/IndianShopping, r/Meesho, and r/india, filtered by keywords: delivery, refund, fake, return, cancel, fraud, quality
- Reddit posts provide pre-purchase intent signal and complaint discussions not captured in post-purchase Play Store reviews
- Add a source column to distinguish Google Play vs Reddit records

1c. Initial Cleaning

- Drop reviews with content length under 10 characters (meaningless ratings like 'ok', '.')
- Normalise timestamps to a consistent datetime format for temporal analysis
- Save merged dataset as raw_reviews.csv with ~2,700 rows

Step 2: Text Preprocessing & Feature Engineering

Preprocessing is deliberately light — the goal is to preserve linguistic signal, especially Hinglish text, rather than over-clean it.

- Lowercase all text; strip URLs, HTML tags, and repeated special characters

- Retain Hinglish words intact: do not transliterate or remove Hindi-origin words. The multilingual SBERT model handles them natively
- Detect language per review using langdetect — add a lang column (en, hi, other)
- Engineer derived features for later analysis: review_length (word count), has_photo (thumbsUpCount > 10 proxy), is_hinglish (lang=en but contains Hindi tokens), star_bucket (1-2 = complaint, 3 = mixed, 4-5 = positive)
- Build a TF-IDF matrix (max_features=5000, ngram_range=(1,2)) in parallel — used only for cluster keyword extraction, not for clustering itself
- Save cleaned dataset as cleaned_reviews.csv

Step 3: Multilingual Sentence-BERT Embeddings

This is the most critical step. The choice of embedding model determines whether Hindi and Hinglish reviews cluster with semantically similar English reviews or form isolated language-based artefacts.

- **Model:** Load paraphrase-multilingual-MiniLM-L12-v2 from the sentence-transformers library. This model supports 50+ languages including Hindi and produces 384-dimensional embeddings
- **Encoding:** Encode all ~2,700 review texts in batches of 64. Runtime: approximately 4 minutes on CPU. Each review becomes a 384-dimensional vector in a shared multilingual semantic space
- **Validation:** Verify semantic alignment: compute cosine similarity between known complaint reviews and known positive reviews. Complaints should cluster (similarity > 0.7) while cross-category similarity should be low (< 0.4)
- **Persistence:** Save embeddings as embeddings.npy (numpy array, ~8MB). This artifact is saved to disk and never recomputed — all downstream stages load it directly

Step 4: Dimensionality Reduction

Raw 384-dimensional SBERT embeddings cannot be clustered directly with K-Means because Euclidean distance is unreliable in high dimensions (the curse of dimensionality). A two-stage reduction pipeline brings the data into a clusterable space while preserving semantic structure.

4a. PCA — Noise Reduction

- Apply PCA to reduce from 384 to 50 dimensions, retaining approximately 85% of variance
- PCA at this stage acts primarily as a denoising step — it removes low-variance dimensions that are dominated by noise rather than semantic content
- Plot a scree curve to verify that 50 components is the right cutoff

4b. UMAP — Clustering Preparation

- Apply UMAP (n_components=10, n_neighbors=15, min_dist=0.1) on the PCA-50 output
- UMAP preserves local neighbourhood structure, which PCA does not — this is what makes text clusters tight and well-separated

- The resulting 10-dimensional UMAP representation is the input to all clustering algorithms

4c. t-SNE — Visualisation Only

- Apply t-SNE (perplexity=30, n_iter=1000) on UMAP-10 to produce a 2D scatter plot
- t-SNE is used purely for visualisation — never for clustering. It exaggerates local structure and is not suitable as a clustering input
- Produce two t-SNE plots: one coloured by cluster label, one coloured by platform. The second reveals whether complaint clusters are platform-agnostic or platform-specific

Step 5: Unsupervised Clustering — Complaint Archetype Discovery

Clustering on the UMAP-10 representation discovers complaint archetypes without any predefined categories or labels. The two algorithms are complementary: K-Means partitions every review into exactly k groups, while HDBSCAN finds natural density clusters and explicitly marks outliers.

5a. K-Means

- Run K-Means for k=3 through k=12. Select optimal k using the Elbow method (inertia curve) and Silhouette score. Expect k=6–8 on this corpus
- Expected complaint archetypes: fake/counterfeit product, delivery failure, payment & refund fraud, app crash/UX bug, size or quality mismatch, and positive experience
- Profile each cluster: compute mean star rating, top-10 TF-IDF keywords, platform distribution, and temporal distribution per cluster

5b. HDBSCAN

- Run HDBSCAN with min_cluster_size=30 on UMAP-10. This finds clusters of varying density without requiring k to be specified
- Reviews labelled -1 (noise) by HDBSCAN are structurally anomalous — they do not fit any cluster. Flag these immediately as seed anomalies for Stage 6
- Compare HDBSCAN cluster boundaries against K-Means clusters using Adjusted Rand Index to assess agreement

5c. Cluster Labelling

- For each K-Means cluster, extract the top-10 TF-IDF terms. Use these to assign a human-readable name: for example, a cluster with top terms 'delivery days late tracking order received' is labelled Delivery Failure
- Cross-tabulate: cluster label x platform x star_bucket. This produces the core business insight table

Step 6: Anomaly Detection — Surfacing Critical Complaints

Anomaly detection identifies individual reviews that are structurally unusual relative to the corpus — reviews that describe incidents severe or unusual enough that no cluster fully captured them. These are the highest-priority complaints for any platform operations team.

6a. Isolation Forest

- Fit Isolation Forest (contamination=0.05, n_estimators=200) on UMAP-10 embeddings
- Extracts an anomaly score for every review. Top 5% by score = flagged as critical
- Isolation Forest works by randomly partitioning the feature space — anomalous points require fewer partitions to isolate, hence lower path length = higher anomaly score

6b. One-Class SVM

- Train One-Class SVM (kernel=rbf, nu=0.05) on reviews in the Positive Experience cluster only — this defines the normal manifold
- Score all remaining reviews. Reviews farthest from the positive manifold are high-risk complaints

6c. Tier 1 Critical Composite Flag

- Reviews flagged by BOTH Isolation Forest AND labelled noise by HDBSCAN are assigned Tier 1 Critical status
- Manually inspect 20 Tier 1 Critical reviews to validate that the anomaly signal is semantically meaningful and not a data artefact
- Build the headline finding: platform x cluster x Tier 1 Critical count — which platform has the most critical unresolved complaint anomalies?

6d. Temporal Spike Detection

- Bin reviews by calendar week using the timestamp column. Compute complaint volume per cluster per week per platform
- Apply a rolling z-score (window=4 weeks) on the weekly count series. Z-score > 2.5 = complaint spike
- A spike means: a platform received abnormally high complaint volume in that cluster that week — a signal of a real operational incident (e.g. a warehouse failure, payment gateway outage, or festive sale quality drop)

Step 7: Streamlit Dashboard Deployment

The Streamlit dashboard makes all analytical outputs accessible to a non-technical audience in real time. The live refresh capability is the project's differentiating demo feature.

- Page 1 — Live Pulse: KPI cards (total reviews ingested, platforms monitored, critical alerts count, top complaint cluster today). A Refresh button re-runs the full ingestion pipeline and updates all pages
- Page 2 — Complaint Landscape: Interactive Plotly t-SNE scatter coloured by cluster label. Hover shows full review text. Sidebar filters by platform, star rating, and date range

- Page 3 — Spike Tracker: Plotly line chart of complaint volume per cluster over time, with spike weeks marked as red dots. Platform selector in sidebar
- Page 4 — Critical Alerts: Ranked table of Tier 1 Critical reviews sorted by anomaly score. Columns: platform, cluster, review text (truncated), anomaly score, date. Red/amber/green severity badge
- Model artifacts persisted with joblib (umap_model.pkl, kmeans.pkl, isoforest.pkl). All heavy computations cached with st.cache_data and st.cache_resource
- Deployed to Streamlit Community Cloud for a public URL suitable for resume and GitHub README

4. Full Technology Stack

Component	Technology / Tool	Purpose
Data Ingestion	google-play-scraper v1.2+	Live review + metadata from Google Play Store (no auth)
Data Ingestion	PRAW (Python Reddit API Wrapper)	Live Reddit posts from r/IndianShopping
Data Processing	pandas, numpy	Dataframe operations, feature engineering, merging
Language Detection	langdetect	Detect English vs Hindi vs mixed reviews
NLP Embeddings	sentence-transformers	Load and run multilingual Sentence-BERT model
NLP Model	paraphrase-multilingual-MiniLM-L12-v2	384-dim multilingual embeddings (50+ languages incl. Hindi)
Text Features	scikit-learn TfidfVectorizer	Cluster keyword extraction and labelling
Dim. Reduction	scikit-learn PCA	Noise reduction: 384-dim to 50-dim
Dim. Reduction	umap-learn	Clustering prep: 50-dim to 10-dim (UMAP)
Dim. Reduction	scikit-learn TSNE	2D visualisation only (not clustering)
Clustering	scikit-learn KMeans	Partition-based clustering, k=6-8
Clustering	hdbscan	Density-based clustering + noise/outlier identification
Anomaly Detection	scikit-learn IsolationForest	Structural anomaly scoring (contamination=0.05)
Anomaly Detection	scikit-learn OneClassSVM	Manifold-based anomaly scoring
Model Persistence	joblib	Save and load trained model artifacts
Visualisation	plotly.express	Interactive t-SNE scatter, time-series charts

Dashboard	streamlit	4-page live dashboard with refresh button
Deployment	Streamlit Community Cloud	Free public hosting, public URL for resume
Version Control	GitHub	Repository with notebooks, src, app.py, requirements.txt

5. Resume Bullet — STAR Method

The STAR method (Situation, Task, Action, Result) structures project descriptions so they communicate business context, your specific contribution, technical choices, and measurable outcomes — exactly what recruiters at consulting firms, MNCs, and product companies look for.

STAR Breakdown

Situation	Indian e-commerce platforms (Myntra, Meesho, Nykaa, Flipkart) receive millions of unstructured user reviews across delivery, product quality, and fraud categories — with no automated system to prioritise or categorise complaints in real time and no labelled training data available.
Task	Build an end-to-end unsupervised complaint intelligence pipeline that ingests live review data, discovers latent complaint patterns without predefined labels, detects anomalous critical complaints, and surfaces actionable insights through an interactive dashboard.
Action	Developed a multimodal NLP pipeline: live ingestion of 2,500+ reviews from 5 Indian apps via Google Play API and Reddit PRAW; multilingual Sentence-BERT embeddings (Hinglish-aware); PCA + UMAP dimensionality reduction; K-Means and HDBSCAN complaint archetype discovery; Isolation Forest + One-Class SVM anomaly detection; temporal spike detection via rolling z-score; deployed as a live-refreshing 4-page Streamlit dashboard.
Result	Identified 6-8 distinct complaint archetypes across 5 platforms with no labelled data; flagged top 5% Tier 1 Critical complaints with cross-platform breakdown; discovered temporal complaint spikes correlating with known operational incidents; live dashboard deployed at public URL.

Final Resume Bullet (Copy-Paste Ready)

Built an unsupervised Complaint Intelligence Engine for Indian e-commerce — ingested 2,500+ live reviews from Myntra, Meesho, Nykaa, Flipkart, and Amazon India via Google Play API + Reddit PRAW; developed Hinglish-aware NLP pipeline using multilingual Sentence-BERT (384-dim), PCA + UMAP dimensionality reduction, and K-Means/HDBSCAN clustering to discover 7

complaint archetypes with zero labelled data; applied Isolation Forest + One-Class SVM anomaly detection to surface Tier 1 Critical complaints and temporal spike detection via rolling z-score; deployed live-refreshing 4-page Streamlit dashboard to public URL with cross-platform complaint comparison.

Short Version (for space-constrained resumes)

Complaint Intelligence Engine | Python, Sentence-BERT, HDBSCAN, Isolation Forest, Streamlit — Unsupervised NLP pipeline ingesting live Google Play + Reddit reviews from 5 Indian e-commerce apps; discovered 7 complaint archetypes via multilingual embeddings + UMAP + K-Means; anomaly detection flagged Tier 1 Critical complaints; live dashboard deployed to Streamlit Cloud.

6. Anticipated Examiner Questions

1. **Why UMAP instead of just PCA for clustering?** — UMAP preserves local neighbourhood structure (which points are near each other) in the low-dimensional space, whereas PCA preserves global variance (which dimensions explain the most spread). For text clustering, local structure matters more — similar complaints should be adjacent in the reduced space. PCA is used first only to denoise the embeddings before UMAP.
2. **Why use both K-Means and HDBSCAN?** — HDBSCAN finds clusters of varying density without requiring k to be prespecified, and explicitly marks points that do not belong to any cluster as noise (label = -1). K-Means forces every point into exactly k clusters regardless of density. In a complaint corpus, some complaints are common and form tight clusters; others are rare and structural outliers. HDBSCAN noise points become our anomaly seeds.
3. **Why cluster on UMAP but visualise with t-SNE?** — t-SNE is purely a visualisation tool — it distorts global distances to exaggerate local structure, which makes it excellent for 2D plots but unreliable for measuring actual similarity. Clustering on t-SNE output would produce results dependent on random seed, perplexity, and iteration count rather than the actual embedding geometry. We cluster on UMAP-10 and visualise with t-SNE-2D.
4. **How does Isolation Forest detect anomalies?** — Isolation Forest isolates anomalies by randomly partitioning the feature space using binary trees. Anomalous points, being structurally different from the majority, require fewer random partitions to be isolated — resulting in shorter average path lengths. The anomaly score is inversely proportional to path length. It requires no labels and scales well to high dimensions.

5. **What is the rolling z-score spike detection?** — The rolling z-score computes, for each week, how many standard deviations the complaint count is above the 4-week rolling mean. A z-score above 2.5 indicates a complaint volume that is statistically unusual for that platform and cluster. This detects operational incidents — a platform outage, a bad batch of products, or a failed festive sale — reflected in sudden complaint surges.